

정보보안 이론과 실제

3판

암호, 접근제어, 네트워크 보안, 소프트웨어

[강의교안 이용 안내]

- 본 강의교안의 저작권은 한빛아카데미(주)에 있습니다.
- 이 자료를 무단으로 전제하거나 배포할 경우 저작권법 136조에 의거하여 처벌을 받을 수 있습니다.



정보보안 이론과 실제

3판

암호, 접근제어, 네트워크 보안, 소프트웨어

Chapter 05 암호화 해시 함수 및 기타

차례

5.1 개요

5.2 암호화 해시 함수란?

5.3 생일 문제

5.4 생일 공격

5.5 비암호화 해시

5.6 SHA-3

5.7 HMAC

5.8 암호 해시 응용 프로그램

5.9 기타 암호 관련 주제

5.10 요약

5.1 개요

■ 이 장에서 살펴볼 내용

- 다양한 정보보안 환경에서 유용한 암호화 해시 함수
- 암호화 해시 함수의 표준 용도인 디지털 서명과 해시된 MAC
- 해시 함수를 이용한 응용 프로그램인 온라인 입찰과 블록체인

■ 이 장에서 이야기할 몇 가지 암호화 관련 주제

- 비밀 공유(시각적 암호화와 관련된 주제 포함)
- 암호화 난수
- 정보 은닉(스테가노그래피와 디지털 워터마킹) 등

5.2 암호화 해시 함수란?

■ 암호화 해시 함수 $h(x)$ 는 다음 특성을 모두 만족해야 함

■ 압축

- 입력 x 의 크기와 상관없이 출력의 길이 $y=h(x)$ 는 항상 작아야 함

■ 효율성

- 어떤 입력 x 에 대해서든 $h(x)$ 를 계산하는 것이 쉬워야 함

■ 단방향

- 어떤 값 y 가 주어진다면 $h(x)=y$ 에서 x 의 값을 찾는 계산이 어려워야 함

■ 약한 충돌 방지

- 주어진 x 와 $h(x)$ 에 대해 $h(y)=h(x)$ 를 만족하면서 $y \neq x$ 인 y 를 찾는 것이 어려워야 함

■ 강한 충돌 방지

- 해시 함수를 써서 똑같은 출력이 나오는 똑같은 두 개의 입력을 찾을 수 없어야 함

5.2 암호화 해시 함수란?

- 해시 함수는 보안에 굉장히 유용
- 해시 함수의 중요한 한 가지 용도는 디지털 서명을 계산하는 것
- 예) 앨리스가 암호화 해시 함수를 가지고 있음
 - $h(M)$ 은 파일 M 에 대한 '지문' 역할을 할 수 있음
 - $h(M)$ 은 M 보다 훨씬 작지만 M 을 식별
 - M' 이 M 과 다르다면, 그것이 한 비트일지라도 해시는 달라짐
 - 충돌 방지 특성은 M 을 $h(M)=h(M')$ 과 같이 다른 메시지 M' 으로 대체하는 것이 불가능하다는 것을 의미
 - [그림 5-1]처럼 앨리스는 밥에게 M 과 S 를 보낼 수 있음
 - 밥은 M 을 해시하고 앨리스의 공개키를 S 에 적용한 값과 비교하여 디지털 서명을 확인

밥은 $h(M)=\{S\}_{\text{앨리스}}$ 라고 확인

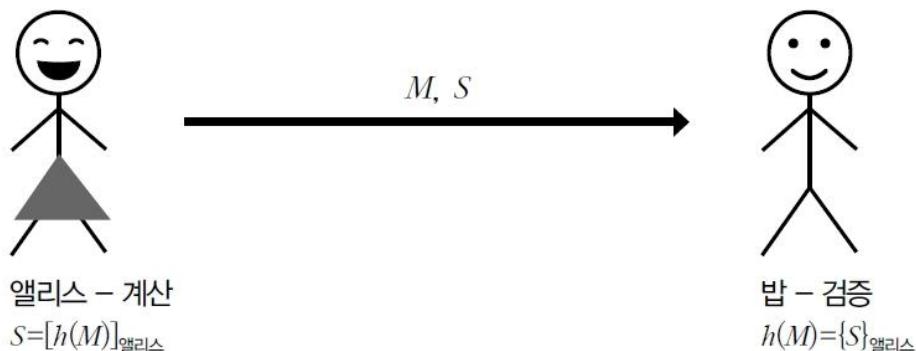


그림 5-1 디지털 서명을 하는 더 나은 방법

5.3 생일 문제

■ 예) 당신이 N 명의 사람들과 함께 있다

- 당신과 생일이 같은 사람을 적어도 한 명 찾기 전에 N 은 얼마나 커야 할까?

$$P(\text{당신과 같은 생일}) = 1 - (364/365)^N$$

- 이 값을 $1/2$ 과 같다고 하고 N 을 구하면 $N=253$
- 두 사람 이상이 같은 생일을 가지기 위해서는 N 이 얼마나 커야 하는가?
 - 여수의 가능성을 풀어서 그 결과에서 1을 빼는 것이 더 쉬움
 - 방 안에 있는 사람들 N 명을 1, 2, 3, ... N 이라고 함
 - 모든 생일이 같다고 가정해보면 확률 결과는 다음과 같음
$$P(N\text{명의 사람 중 두 사람 이상이 생일이 같다})$$
$$= 1 - 365/365 \cdot 364/365 \cdot 363/365 \dots (365 - N + 1)/365$$
 - 이 값을 $1/2$ 과 같다고 두고 N 을 구하면 $N \approx 23$ 을 구하게 됨

5.3 생일 문제

- 생일 문제는 종종 생일의 역설이라고도 함 - 하지만, 결과는 덜 역설적
 - 이 문제에서는 모든 쌍의 사람들의 생일을 비교함
 - 방 안에 N 명의 사람들이 있으면, 비교의 수는 $N(N-1)/2 \approx N^2$
 - 다른 생일이 365개가 있으므로, 대략 $N^2=365$ 또는 $N=\sqrt{365} \approx 19$ 일 때 쌍을 찾기를 기대 → 생일 역설은 그렇게 역설적이지 않음
- 생일이 암호화 해시 함수와 무슨 연관이 있는가?
- 생일 문제는 약 $2^{N/2}$ 의 무작위 노동력으로 N 비트 출력을 생성하는 안전한 해시도 해독할 수 있다는 것을 암시

5.4 생일 공격

■ 예) 해시 함수 h 가 n -비트 출력을 생성한다

■ 원칙적으로 트루디는 다음과 같이 생일 공격을 시행함

- ① 트루디는 앨리스가 서명하지 않을 '악의적인' 메시지 E 를 선택
- ② 그리고 트루디는 앨리스가 확실히 설명할 만한 무해한 메시지 I 도 생성
- ③ 그런 후에 트루디는 무해한 메시지를 약간 편집하여 $2^{n/2}$ 의 변수를 생성
- ④ 이와 유사하게, 트루디는 악의적인 메시지 E_j 의 $2^{n/2}$ 변수를 생성
- ⑤ 트루디는 모든 악의적인 메시지 E_j 와 모든 무해한 메시지 I_k 를 해시

충돌을 고려하여 트루디는 I_k 를 앨리스에게 보내고 서명을 요청

앨리스는 거기에 서명을 하고 I_k 와 $[h(I_k)]_{\text{앨리스}}$ 를 트루디에게 보냄

$h(E_j)=h(I_k)$ 이므로, $[h(E_j)]_{\text{앨리스}}=[h(I_k)]_{\text{앨리스}}$ 가 되고,

결과적으로 트루디는 실질적으로 악의적 메시지 E_j 에 대한 앨리스의 서명을 획득하게 됨

5.5 비암호화 해시

- 많은 비암호화 해시는 암호화 응용 프로그램에 적합하지 않음

$$X=(X_0, X_1, X_2, \dots, X_{n-1})$$

여기서 각각의 X_i 는 바이트

$$h(X)=(X_0+X_1+X_2+\dots+X_{n-1}) \pmod{256}$$

- 두 개의 바이트를 교환하면 항상 충돌이 일어남

$$h(10101010, 00001111)=h(00001111, 10101010)=10111001$$

$$h(X)=(nX_0+(n-1)X_1+(n-2)X_2+\dots+2X_{n-2}+X_{n-1}) \pmod{256}$$

- 이 해시는 암호학적 측면에서 안전할까? 두 바이트를 서로 바꾼 결과

$$h(10101010, 00001111) \neq h(00001111, 10101010)$$

- 이 함수도 상대적으로 쉽게 충돌할 수 있음

$$h(00000001, 00001111)=h(00000000, 00010001)=00010001$$

5.5 비암호화 해시

■ 암호화 해시로 오해 받는 비암호화 해시

- 순환 중복 검사(CRC)
 - CRC 계산은 CRC '해시' 값으로 행동하는 나머지가 있는 긴 나눗셈
 - 일반적인 긴 나눗셈과 다르게 CRC에서는 빼기 대신에 XOR 연산을 사용
- WEP는 암호 무결성 검사가 필요한 곳에 실수로 CRC 체크섬을 사용
 - 프로토콜 공격자에게 문을 열어주는 것과 같음

5.6 SHA-3

■ SHA-3

- SHA-1 알고리즘과 SHA-2 알고리즘 제품군을 대체하기 위해 설계된 미국 정부 표준 알고리즘
- SHA-3는 케ccak(keccak)으로 알려진 이전에 개발된 기술에 그 기반을 두고 있음
- 이전에 개발된 대부분의 암호화 해시 함수는 MD5와 SHA-1을 포함하여 머클-담가드(Merkle-Damgard) 구성에 기반을 두고 있음
 - 약점: 길이 확장 공격(length extension attack)
- 케ccak 알고리즘은 메시지를 블록으로 해시('스폰지' 기술을 사용)
 - 스폰지의 '흡수' 과정 동안 각 블록은 차례로 현재의 내부 상태의 순열로 XOR 연산을 실시
 - 전체 메시지가 흡수되고 나면, '압축' 단계에서는 해시 값을 구성하는 내부 상태의 비트를 추출

5.6 SHA-3

■ SHA-3 스펀지 기술

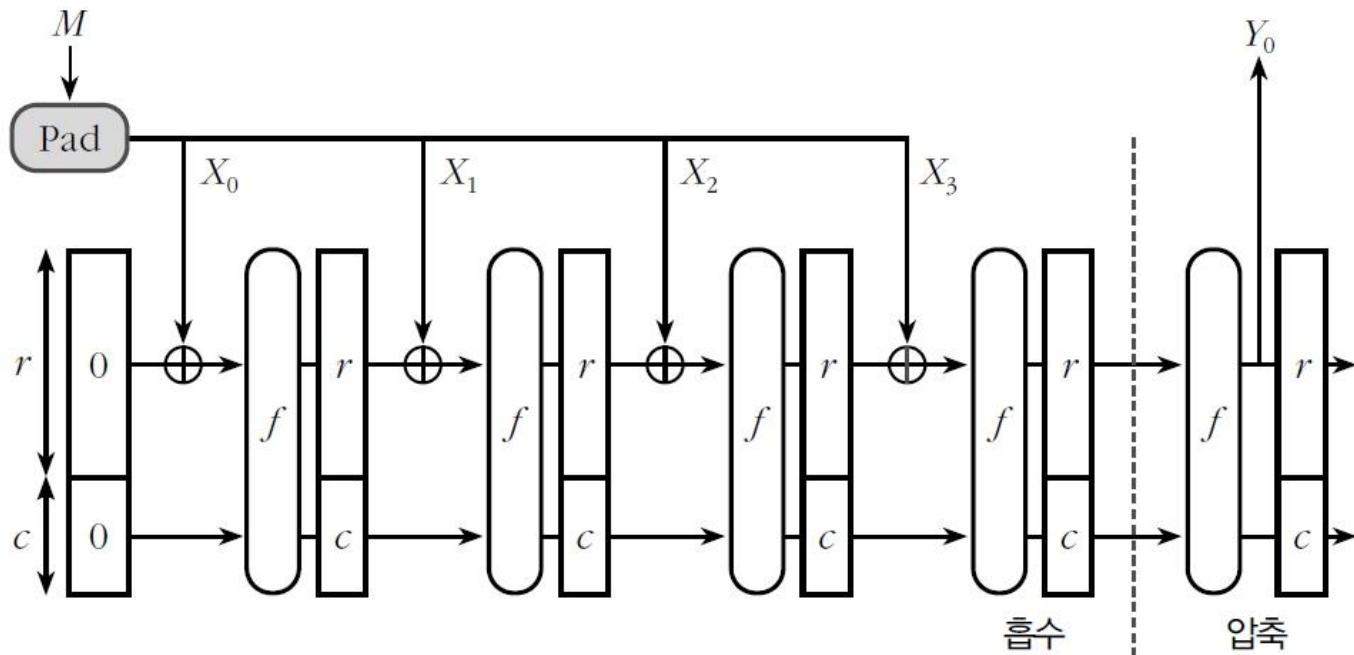


그림 5-2 SHA-3 '스폰지'

5.6 SHA-3

- 스펀지 기술의 보안성은(효율성뿐만 아니라) f 의 선택에 달려 있음
- SHA-3에서 f 함수는 24라운드로 구현되며 각 라운드는 5단계로 이루어져 있음
- SHA-3의 블록 크기는 1600비트
- 효율성을 위해 내부 상태는 64비트 단어의 5×5 그리드로 처리되며 각 단어는 '레인'으로 지칭
- 이 5×5 그리드의 레인 (i,j) 에 있는 64비트 단어를 $A[i,j]$ 로 표시

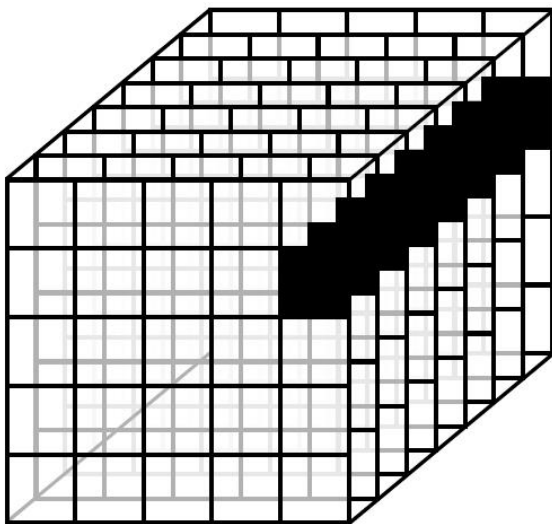


그림 5-3 SHA-3 상태 $A[x,y]$ (64비트 단어의 5×5 배열)

5.6 SHA-3

■ SHA-3의 i 라운드에서의 단계

표 5-1 SHA-3의 i 라운드에서의 단계

```
//  $\theta$  단계
for  $x = 0$  to  $4$ 
     $C[x] = A[x,0] \oplus A[x,1] \oplus A[x,2] \oplus A[x,3] \oplus A[x,4]$ 
next  $x$ 
for  $x = 0$  to  $4$ 
     $D[x] = C[x-1] \oplus (C[x+1] \lll 1)$ 
next  $x$ 
for  $x = 0$  to  $4$ 
    for  $y = 0$  to  $4$ 
         $A[x,y] = A[x,y] \oplus D[x]$ 
    next  $y$ 
next  $x$ 
//  $\rho, \pi$  단계
for  $x = 0$  to  $4$ 
    for  $y = 0$  to  $4$ 
         $B[y, 2x+3y] = (A[x,y] \lll r[x,y])$ 
    next  $y$ 
next  $x$ 
//  $\chi$  단계
for  $x = 0$  to  $4$ 
    for  $y = 0$  to  $4$ 
         $A[x,y] = B[x,y] \oplus ((\sim B[x+1,y]) \& B[x+2,y])$ 
    next  $y$ 
next  $x$ 
//  $\iota$  단계
 $A[0,0] = A[0,0] \oplus R_i$ 
```

5.7 HMAC

- 메시지 무결성을 위해 메시지 인증 코드인 MAC를 계산할 수 있음
- 메시지 무결성을 검증하기 위해서는 해시를 사용할 수 있어야 함
- 앨리스가 단순히 $h(M)$ 을 계산하거나 또는 M 과 $h(M)$ 을 밥에게 보냄으로써 M 의 무결성을 보호할 수 있는가?
 - 가장 확실한 방법은 앨리스가 해시 값을 대칭 암호 $E(h(M), K)$ 로 암호화해서 이것을 밥에게 보내는 것
- 실제로는 해시된 MAC인 HMAC를 계산하는 데 조금 다른 방법을 사용
 - 해시를 암호화하는 대신에 해시를 계산할 때 키를 M 에 직접 혼합
 - 키를 메시지 시작 부분에 더하기: $h(K, M)$
 - 키를 메시지 뒤에 첨부하기: $h(M, K)$

5.7 HMAC

- HMAC를 $h(K, M)$ 으로 계산하는 것을 선택했다고 가정

- 대부분의 암호화 해시는 메시지를 블록으로 해시
- MD5와 SHA-1 블록 크기는 512비트
- 만약 $M=(B_1, B_2)$ 라면 여기서 B_i 는 512비트
- 그러면 함수 F 에 대해 다음이 성립(A 는 고정된 초기 상수)

$$h(M)=F(F(A, B_1), B_2)=F(h(B_1), B_2) \quad (5.1)$$

- 만약 트루디가 $M'=(M, X)$ 가 되도록 M' 을 선택한다면 식 (5.1)을 사용해 $h(K, M)$ 에서 $h(K, M')$ 을 찾을 수 있음. (5.2)가 성립하기 때문

$$h(K, M')=h(K, M, X)=F(h(K, M), X) \quad (5.2)$$

- 이번에는 $h(M, K)$ 를 선택했다고 가정

- 해시 함수 h 에 대해 알려진 충돌이 있어서 M' 이 $h(M')=h(M)$ 으로 존재한다면 식 (5.1)에 의해 다음과 같이 됨

$$h(M, K)=F(h(M), K)=F(h(M'), K)=h(M', K) \quad (5.3)$$

- 여기서 M 과 M' 은 각각 블록 크기의 배수. 만약에 충돌이 일어나면 해시 함수는 안전하지 않은 것으로 봄

5.7 HMAC

■ HMAC를 계산하는 승인된 방법

- B 를 해시의 바이트 단위 블록 길이라고 가정
- 현재 많이 사용하는 해시 함수에서는 $B=64$
 - $\text{ipad}=0x36$ 을 B 번 반복
 - $\text{opad}=0x5C$ 를 B 번 반복
- M 의 HMAC는 다음과 같이 계산
 - $\text{HMAC}(M, K) = h(K \oplus \text{opad}, h(K \oplus \text{ipad}, M))$

5.8 암호 해시 응용 프로그램

■ 온라인 입찰

- 앨리스, 밥, 척이 입찰하고자 하는 경매 물건이 온라인에 나왔다고 가정
 - 앨리스는 10달러에 입찰을 하고 밥은 12달러에 입찰을 한다고 가정 → 만약 척이 두 사람의 입찰 금액을 입찰 마감 전에 미리 안다면 12달러 1센트를 적어내서 입찰에 성공
 - ↳ 나중에 입찰하는 것이 유리할 수 있기 때문에 어느 누구도 입찰 금액을 먼저 적어 내려고 하지 않음
 - 이러한 우려를 없애기 위해 각 입찰자는 자신의 입찰 금액을 결정(비밀)
 - 앨리스는 $h(A)$ 를 제출하고 밥은 $h(B)$ 를, 척은 $h(C)$ 를 제출
 - 세 사람이 세 개의 해시 입찰을 제출하면 해시 값을 온라인에 게시해 모두가 볼 수 있음
- 이 방법이 입찰을 직접 제출하는 것보다 나은 이유는 무엇일까?
 - 해시 값은 입찰 자체에 대한 정보는 노출시키지 않고 입찰자를 그들의 원래 입찰과 함께 묶어버림
 - 해시 값을 한 번 제출하고 나면 입찰을 바꾸는 것이 불가능하다면 입찰을 직접 제출했을 때 발생하는 문제를 예방

5.8 암호 해시 응용 프로그램

■ 블록 체인(blockchain)

- 비트코인과 같이 암호화폐 영역에서 유명해짐

■ 디지캐시(DigiCash)

- 가장 초기 형태이자 가장 잘 알려진 전자현금
- 디지캐시를 이루는 근본 아이디어는 차움의 블라인드 서명을 사용한다는 것
- 완전히 탈중앙화된 전자화폐 개념은 훨씬 최근에 발명됨
- 탈중앙화된 전자화폐 보안에 블록체인을 사용하는 것은 몇 가지 뛰어난 암호학을 기반으로 함

5.8 암호 해시 응용 프로그램

■ 원장

- 금융거래의 장부로 정의
- 미국 달러를 포함하는 모든 거래가 원장에 기록된다고 가정

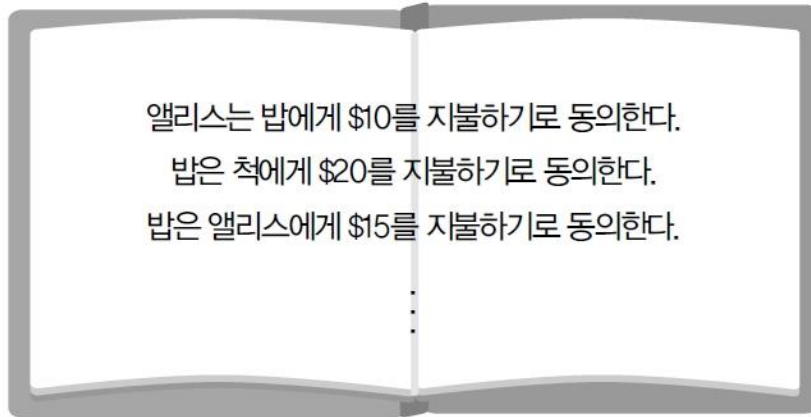


그림 5-4 원장 예시

- 블록체인을 사용하여 분산원장을 생성하고 유지할 수 있음
- 탈중앙화된 전자화폐는 암호 기술에 의지하기 때문에 이를 암호화폐라고 부를 것임

5.8 암호 해시 응용 프로그램

■ 암호화폐의 장점과 단점

■ 장점

- 완전히 탈중앙화된 암호화폐는 그 어떤 중앙 권한 즉, 은행이나 정부도 필요하지 않으므로 정치적 영향력이나 실물 화폐에 영향을 주는 다양한 조작으로부터 자유로움

■ 단점

- 범죄자들이 전자화폐 거래를 선호할 수 있어 범죄 수단이 될 가능
- 정부는 통화를 조작할 수 있는 능력을 잃을 수 있어서 정책을 수립하고 이행하는 데 어려움을 겪을 수 있음

■ 탈중앙화된 전자화폐를 구성하기 위해서는 작업 단위가 필요

- 암호 해시를 사용

■ 암호 해시 함수 $h(x)$ 가 N 비트 출력을 생성해낸다고 가정

- 어떤 입력 R 에 대해 $0 \leq h(R) < 2^N$ 이며, 입력 $R_1, R_2, R_3, \dots$ 에 대해 그 결과로 나오는 해시 값이 $h(R_1), h(R_2), h(R_3), \dots$ 로 균등하게 배분된다는 점에서 이 범주 안의 모든 해시 값은 동등

5.8 암호 해시 응용 프로그램

■ 분산원장

- 중앙 권한이 없는 분산원장의 경우
 - '앨리스가 밥에게 10달러를 주기로 동의했다'와 같은 항목이 유효한지 어떻게 알 수 있는가?
 - 원장 항목에 디지털 서명을 요구하면 쉽게 해결됨

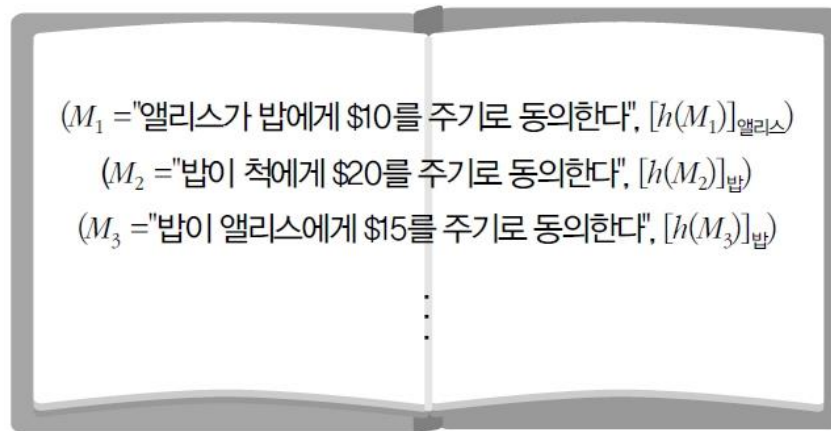


그림 5-5 서명된 원장 항목

- 서명된 전자원장에는 몇 가지 명확한 보안 문제가 남아 있음
 - ↳ 모든 항목은 몇 번을 반복해도 여전히 유효하다는 것

5.8 암호 해시 응용 프로그램

- 다음과 같은 내용이 다섯 번 반복된다고 가정

$(M_1 = \text{"앨리스가 밥에게 \$10를 주기로 동의한다"}, [h(M_1)]_{\text{앨리스}})$

- 실제로는 앨리스가 이 거래 중 하나에만 서명을 했을지라도 앨리스가 밥에게 50달러를 주기로 동의했다는 것처럼 보일 수 있음
- 이 문제를 해결하기 위해, 각 항목에 거래 번호를 추가할 수 있음

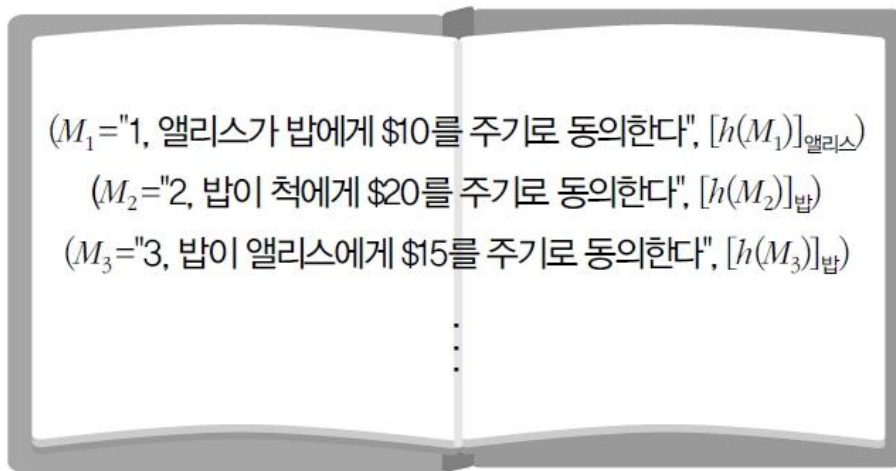


그림 5-6 숫자가 매겨지고 서명된 원장 항목

5.8 암호 해시 응용 프로그램

■ 원장이 가지는 또 다른 문제

- 어떤 사람이 실제로 낼 수 없는 돈을 지불하겠다고 할 수 있다는 것
- 그런 과소비를 막기 위해, 모든 사람들이 장부에 초기 금액을 지불하도록 요구할 수 있고, 그다음 그들이 가진 것보다 더 많이 쓰지 않는지 확인할 수 있음

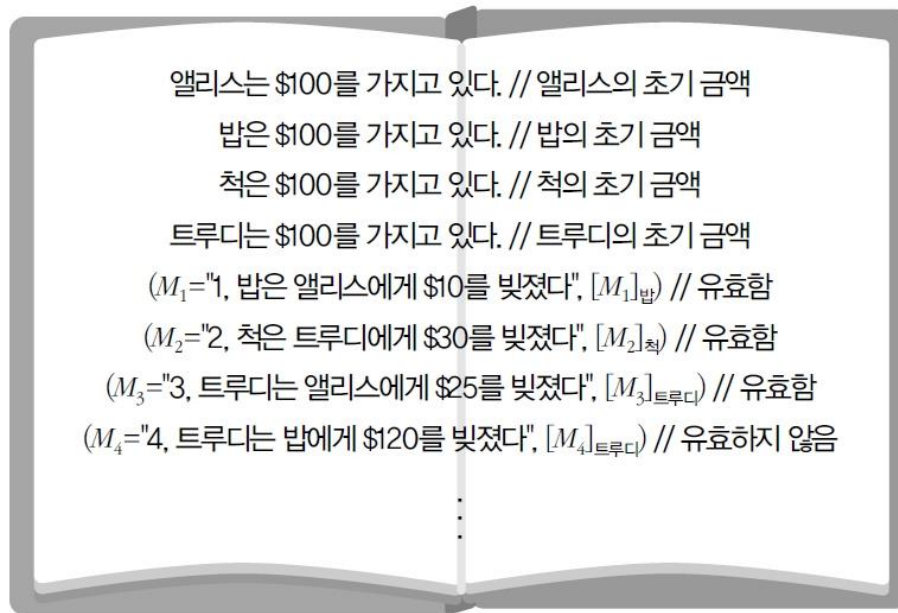


그림 5-7 유효하지 않은 정보가 추가된 원장

5.8 암호 해시 응용 프로그램

- 분산형 원장이 갖는 문제를 해결하기 위한 규칙
 - 거래는 서명되어야만 함
 - 어느 누구도 초과 인출할 수 없음
 - 거래는 모든 사람에게 공개되어야 함
 - ↳ 이 규칙을 적용하더라도 여전히 여러 개의 원장이 언제든지 존재 가능
- 스탬프코인(stampcoin)
 - 모든 화폐는 그것이 암호화폐든 아니든 그 자체를 나타내는 상징이 필요
 - 여기서 논의되는 암호화폐의 상징으로 우리는 \$를 사용

5.8 암호 해시 응용 프로그램

■ 블록체인은 블록이랑 체인이 필요

- 개인 거래를 묶어서 블록(여기서는 B)으로 그룹 짓겠음
- 암호 해시 함수 h 는 N -비트 출력을 생산
- 특정한 m 에 대해 $h(B, R) < 2^m$ 이 되는 숫자 R 을 찾아보기
 - 이것은 $h(B, R)$ 이 적어도 0s를 이끄는 $N-m$ 개를 가지는 것과 같음
 - R 을 찾도록 예상하기 전에 2^{N-m} 에 대해 계산해야 함
 - 주어진 B 에 대해 만약 $h(B, R) < 2^m$ 이 되도록 R 을 생산해낼 수 있다면, 이것은 평균적으로 2^{N-m} 작업의 단위를 수행했음을 확인하는 데 도움이 될 것임
 - 이러한 해시 값은 블록을 검증하는 역할을 함
 - 물론, R 이 주어지면, 단일 해시를 계산하여 그렇게 주장된 유효성을 검증 가능

5.8 암호 해시 응용 프로그램

- 유효한 R 을 찾는 데 필요한 해시를 계산하는 사람을 위한 인센티브는 무엇인가? 정답은 '공짜' 돈
 - 누구든지 제일 먼저 찾는 사람은 1달러인 1스탬프코인을 받게 됨
 - 누구든지 새로운 블록 B 를 만들어낼 수 있지만 유효한 해시를 생산하는 R 을 찾는 데는 많은 노력이 듦

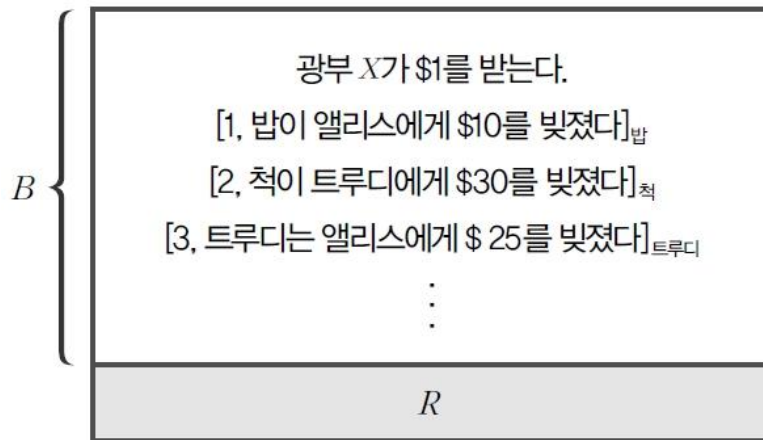


그림 5-8 $h(B,R) < 2^m$ 인 블록 B 와 R

5.8 암호 해시 응용 프로그램

■ 체인은 효율성을 고려한 것

- 단계마다 각각의 블록을 재검증하고 싶지 않기 때문에 블록을 체인으로 묶게 되는 것
- 체인을 생성하기 위해 이전 블록의 해시 값을 현재 블록의 헤더에 둠
- 만약 Y 가 이전 블록의 해시라면 블록 B 를 위한 검증 과정은 $h(Y, B, R) < 2^m$ 이 되도록 R 을 찾고자 함

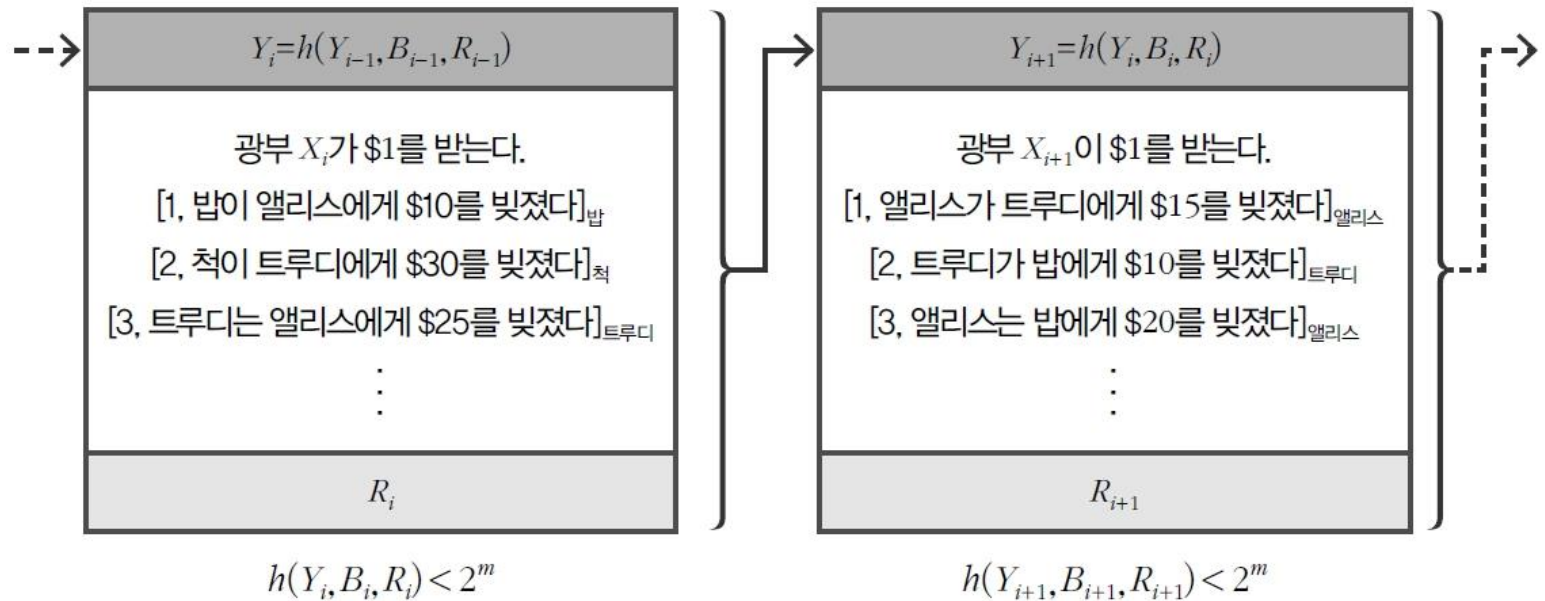


그림 5-9 블록체인의 일부

5.8 암호 해시 응용 프로그램

■ 블록체인 기반의 암호화폐의 주요 요점

- 새로운 거래는 방송된다.
- 광부들은 거래를 모아서 블록으로 만든다.
- 광부들은 유효한 블록 해시를 계산하기 위해 경쟁하는데 이를 통해 새로운 블록이 블록 체인에 통합될 수 있다.
- 유효한 해시를 찾은 광부는 이것을 공개한다.
- 새롭게 채굴된 블록은 모든 거래가 서명되고, 초과 인출이 없고 새로운 블록 해시가 유효하면 받아들여지게 된다.
- 새롭게 받아들여진 블록은 블록체인으로 확장되고 광부들은 새롭게 확장된 블록체인을 계속되는 채굴 계산에 사용하게 된다.

5.8 암호 해시 응용 프로그램

■ 공격 시나리오: 트루디가 100달러를 가지고 있음

- $T = [1, \text{트루디가 앨리스에게 \$100를 지불한다}]_{\text{트루디}}$
 - 트루디는 이것을 오직 앨리스에게만 보냄(트루디의 사적 거래 T)
 - 앨리스가 이 사적 거래를 받아들이면 트루디는 자신의 100달러를 또 마음껏 쓸 수 있음
- 트루디가 사용하는 이러한 이중 소비 공격을 무엇으로 막을 수 있을까?
- 트루디의 공격이 성공할 가능성은 트루디의 통제하에 있는 네트워크의 전체 컴퓨팅 파워(computing power)에 정비례함
 - 블록체인의 가장 최근 블록은 신뢰도가 가장 낮다. 아주 적은 컴퓨팅 파워로도 트루디는 블록을 채굴할 수 있으며 다른 누구보다 먼저 사적 블록체인을 만들 수 있음
- 암호화폐 접근 방식에는 몇 가지 가능한 개선 사항이 존재
 - 네트워크의 컴퓨팅 파워가 증가함에 따라 매개변수 m 에 기반을 둔 해시의 예상되는 수를 조정 가능
 - 머클 트리(merkle trees) 사용을 포함하는 것

5.9 기타 암호 관련 주제

■ 비밀 공유

- 엘리스와 밥은 비밀 s 를 다음과 같은 의미로 공유하고자 함
 - 엘리스와 밥 모두 단독으로는 비밀 s 를 단지 막연히 추측하는 것 이상으로 더 정확하게 알아낼 수는 없음
 - 엘리스와 밥은 함께 비밀 s 를 구할 수 있음
 - 비밀 s 를 획득하기 위해 두 참가자가 모두 협동해야 하기 때문에 이것을 비밀 공유 체계라고 부름

5.9 기타 암호 관련 주제

- 비밀 s 가 실제 숫자라고 가정

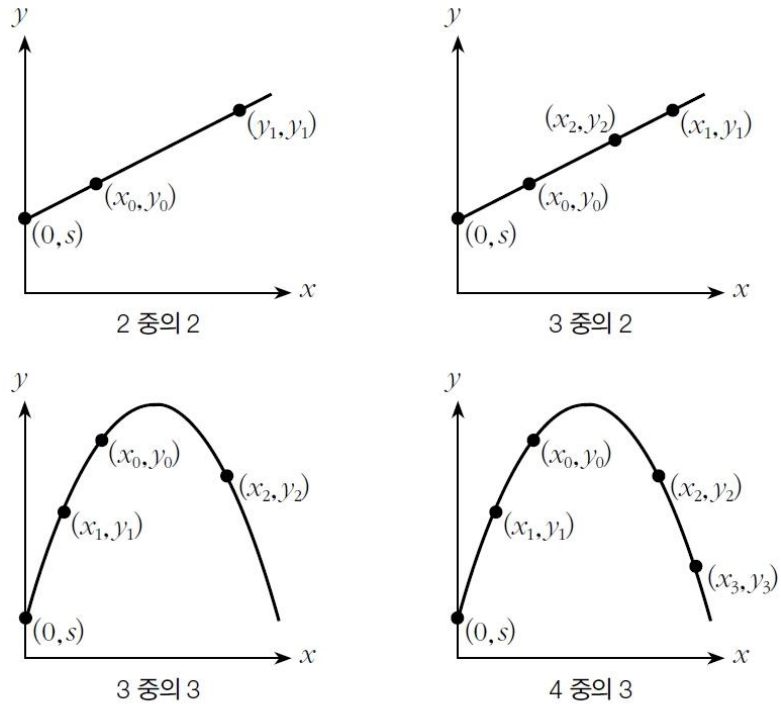


그림 5-10 비밀 공유 체계

5.9 기타 암호 관련 주제

■ 조건부 키

- 비밀 공유가 유용하게 사용될 특정 응용 분야 중 하나는 조건부 키 문제
- 사용자의 키를 공식적인 조건부 기관에 맡기도록 요청했다고 가정
 - 미국 정부는 조건부 키를 촉진하고 심지어 조건부 키를 포함하는 체계 (clipper and capstone)를 개발하고자 함
 - ↳ 조건부 키 아이디어는 많은 비난을 받고 결국 취소됨
 - 조건부 키와 관련된 한 가지 문제는 조건부 기관을 신뢰하지 못할 수도 있다는 것
 - ↳ 여러 개의 조건부 기관을 둬으로써 그리고 사용자가 키를 n 개로 나눠서 n 개 중 m 개를 함께 이용해야만 키를 복구할 수 있게 함으로써 어느 정도 개선 가능
 - 샤미르의 비밀 공유 체계는 이런 조건부 키 체계를 실행하는 데 사용될 수 있음

5.9 기타 암호 관련 주제

■ 시각적 암호

- 나오와 샤미르[89]는 흥미로운 시각적 비밀 공유 체계를 제안
- 예) 흑백 이미지로 두 개의 투명도를 만드는데 하나는 앨리스를 위한 것이고 또 다른 하나는 밥을 위한 것
 - 여기서 뭇 1은 앨리스의 투명도, 뭇 2는 밥의 투명도로 이해할 수 있음

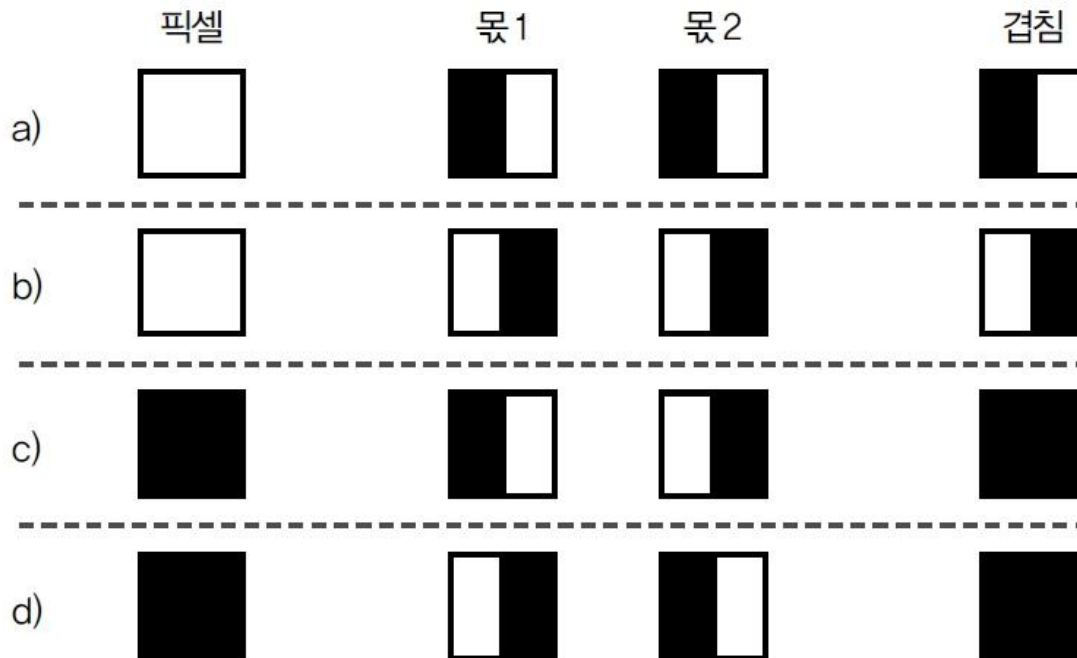


그림 5-11 픽셀을 뭇 1과 뭇 2로 공유

5.9 기타 암호 관련 주제

- 원본 픽셀이 검은색이면 겹치는 뭉은 항상 검은색 픽셀로 나타남
- 원본 픽셀이 흰색이면 겹치는 뭉은 반은 흰색/반은 검은색 픽셀로 나타나며 이것은 회색으로 인식됨
- 이것은 대비의 손실(검은색과 회색 대 검은색과 흰색)로 나타나겠지만 원본 이미지는 여전히 명확하게 구별
- 예) [그림 5-12]에서 엘리스와 밥에 대한 각각의 뭉과 함께 엘리스와 밥의 뭉이 겹쳐져서 생기는 이미지도 보여줌

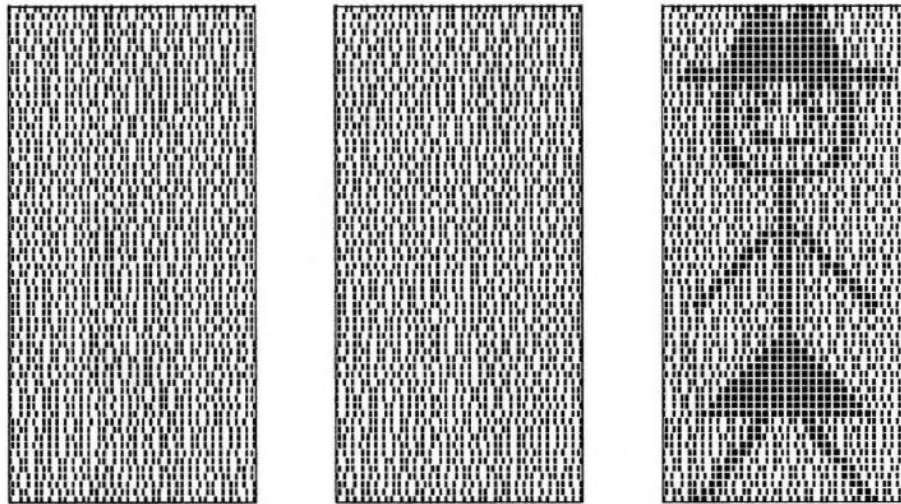


그림 5-12 엘리스의 뭉, 밥의 뭉, 그리고 겹치는 이미지

5.9 기타 암호 관련 주제

■ 난수

- 암호학에서 난수는 대칭키와 RSA 키 쌍, 즉, 무작위로 선택되는 큰 소수들 그리고 디피-헬먼 비밀 지수 등을 생성하는 데 사용되어 옴
- 난수는 시뮬레이션과 다양한 통계 응용 프로그램과 같은 비보안 응용 프로그램에서 사용
 - 난수는 통계적으로 무작위인 특정 분산 요구를 만족해야 함
- 서버가 사용자들을 위한 일련의 대칭키를 생성한다고 가정
 - K_A 는 앨리스의 키, K_B 는 밥의 키, K_C 는 척의 키, K_D 는 데이브의 키
 - 만약 K_D 를 K_A , K_B , K_C 를 통해 알아낼 수 있다면 그 체계의 보안은 무너지게 됨
 - 흔히 사용되는 의사 난수 발생기는 예측 가능

5.9 기타 암호 관련 주제

■ 텍사스 홀덤 포커

- 난수를 생성하는 잘못된 방법을 잘 표현하는 실제 세계 예

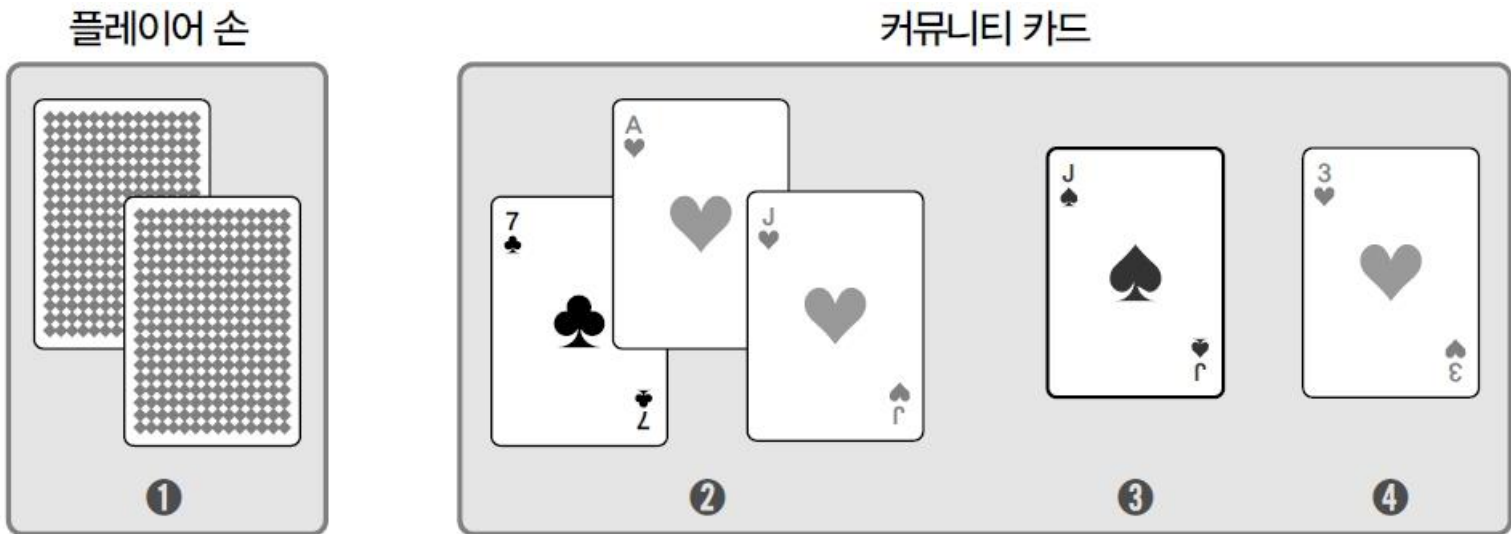


그림 5-13 텍사스 홀덤 포커

5.9 기타 암호 관련 주제

■ 무작위 비트 생성

- 진정한 무작위는 찾기 힘들 뿐만 아니라 정의하기도 힘들
 - 아마도 최선의 방법은 클라우드 새넌이 개발한 엔트로피 개념일 것
- 진정한 무작위의 원천은 확실히 존재
 - 예를 들어, 방사능 물질의 붕괴가 그 무작위가 될 수 있음

5.9 기타 암호 관련 주제

■ 정보 은닉 – 스테가노그래피(steganography)

- '감추어진 기록'이란 의미로 정보가 전송된다는 사실을 숨기려는 시도
- 전장에서 긴 역사를 가지고 있음
- 현대 버전의 스테가노그래피는 사진 파일, 음성 데이터 또는 소프트웨어와 같은 미디어 속에 정보를 은닉하는 것이 포함

■ 워터마크는 투명성과 가시성으로 분류

- **투명성** 사람들 눈에 보이지 않는 워터마크
- **가시성** 일급 기밀 도장처럼 눈에 보이는 워터마크

■ 워터마크는 강한 워터마크와 약한 워터마크로 분류

- **강한 워터마크** 공격을 받아도 읽을 수 있는 워터마크
- **약한 워터마크** 공격을 받으면 파괴되거나 손상되는 워터마크

5.9 기타 암호 관련 주제

■ 스테가노그래피의 형태로 숨어 있는 정보에 접근하는 예①

■ 디지털 이미지에 적용 가능

- [그림 5-14]에서 두 개의 앨리스 이미지를 살펴보기
- 왼쪽의 앨리스는 어떤 숨겨진 정보도 없지만 오른쪽의 앨리스는 낮은 자리 비트의 RGB에 『이상한 나라의 앨리스』 책이 있음
- 보통 사람의 눈에는 어떤 해상도에서든 두 이미지가 동일하게 보임
- 두 이미지의 비트를 비교하면 차이가 있음



(a) 평균 앨리스



(b) 그녀의 책을 숨긴 앨리스

그림 5-14 두 앨리스의 이야기

5.9 기타 암호 관련 주제

■ 스테가노그래피의 형태로 숨어 있는 정보에 접근하는 예②

- 「월러스와 목수」 [15]의 다음과 같은 문구를 담고 있는 HTML

"때가 되었어." 월러스가 말했다.

"많은 것들에 대해 이야기 할:

신발과 배와 실링 왁스에 대해

양배추와 왕에 대해

그리고 왜 바다가 뜨겁게 끓고 있는지

그리고 돼지가 날개를 가지고 있는지 아닌지에 대해."

- HTML에서 RGB 폰트 컬러는 다음과 같은 형태의 태그로 특정됨

`<font color="#rrggbb"> ... </font>`

(여기서 rr은 16진법에서 R의 값을, gg는 G를, bb는 B의 값을 나타냄)

- R, G, B의 낮은 자리 비트가 인식된 색에 영향을 주지 않기 때문에 [표 5-2]의 HTML의 일 부분에서 나타나듯이, 이 비트에 정보를 숨길 수 있음

5.9 기타 암호 관련 주제

- 각 RGB 색의 낮은 자리 비트를 읽으면 '숨겨진' 정보 101 110 101 010 110 101이 나타남

표 5-2 간단한 스테가노그래피 예시

```
<font color="#010001">"때가 되었어,"  
    월러스가 말했다.</font><br>  
<font color="#010100">많은 것들에 대해 이야기 할:</font><br>  
<font color="#010001">신발과 배와 실링 왁스에 대해</font><br>  
<font color="#000100">양배추와 왕들에 대해</font><br>  
<font color="#010100">그리고 왜 바다가 뜨겁게 끓고 있는지</font><br>  
<font color="#010001">그리고 돼지가 날개를 가지고 있는지 아닌지에 대해</font><br>
```

5.10 요약

- 암호화 해시 함수
- SHA-3
- MAC(HMAC)를 계산하는 올바른 방법
- 암호 해시 함수의 응용 프로그램
 - 블록체인 기술, 암호화폐 응용 프로그램
- 샤미르의 비밀 공유 체계
- 나오와 샤미르의 시각적 암호화
- 난수 생성
- 정보 은닉
 - 디지털 스테가노그래피, 디지털 워터마킹

정보보안 이론과 실제

3판

암호, 접근제어, 네트워크 보안, 소프트웨어

감사합니다.